

Raport Tehnic: Sistem Client-Server pentru Managementul Campionatelor

Romaniuc Tudor Grupa 2E3

Facultatea de Informatică "Alexandru I. Cuza" Iași

Abstract. Acest document detaliază arhitectura și implementarea unei aplicații distribuite de tip Client-Server, dezvoltată în limbajul C++, pentru gestionarea competițiilor e-sports. Sistemul permite înregistrarea utilizatorilor, crearea de campionate de către administratori, înscrierea jucătorilor și generarea automată a meciurilor, asigurând persistența datelor prin fișiere text și comunicarea fiabilă prin protocolul TCP.

1 Introducere

Proiectul propune o soluție software pentru coordonarea campionatelor, adresând nevoia unei platforme centralizate care să gestioneze utilizatori și evenimente competitive. Obiectivul principal este asigurarea unui flux coerent de date într-un mediu concurent.

Arhitectura aleasă este de tip Client-Server. Serverul acționează ca autoritate centrală pentru validarea logicii de business (autentificare, permisiuni, generare meciuri) și stocarea datelor, în timp ce clienții interacționează printr-o interfață în linie de comandă (CLI). Implementarea se bazează pe primitivele de sistem POSIX, folosind socket-uri BSD pentru rețea și procese pentru concurență.

2 Tehnologii Aplicate

Implementarea a fost realizată pe un sistem de operare compatibil Linux, utilizând următoarele tehnologii:

- **Protocolul TCP (Transmission Control Protocol):** Aplicația utilizează socket-uri de tip `SOCK_STREAM`. S-a optat pentru TCP în detrimentul UDP deoarece operațiunile critice, precum autentificarea (`LOGIN`) sau actualizarea scorurilor (`SET_SCOR`), necesită o conexiune fiabilă, ordonată și fără pierderi de pachete.
- **Concurență prin Multi-procesare (fork):** Pentru a deservi mai mulți clienți simultan, serverul apelează `fork()` la fiecare conexiune acceptată. Procesul fiu preia comunicarea cu clientul, permițând serverului părinte să revină la starea de ascultare. De asemenea, clientul folosește `fork()` pentru a separa logica de ascultare (primire mesaje asincrone de la server) de cea de citire de la tastatură.

- **Persistența Datelor:** Stocarea datelor se realizează non-volatil în fișiere text delimitate prin caracterul ‘—’ (`db_users.txt`, `db_campionate.txt`, `db_meciuri.txt`). Această abordare simplifică inspecția datelor și este adecvată pentru volumul didactic al proiectului.
- **Simulare Email:** Notificările sunt simulate prin scrierea într-un fișier de log dedicat (`server_emails.txt`), emulând comportamentul unui server SMTP.

3 Structura Aplicației

Sistemul este compus din două entități distincte care comunică prin socket-uri pe portul 2023.

Serverul gestionează starea globală a aplicației. La pornire, acesta încarcă sau creează fișierele de bază de date. Fluxul principal implică bucla infinită de `accept()`, urmată de crearea unui proces fiu care instanțiază structura `UserSesiune` (pentru a reține starea de logare și drepturile de administrator). Funcția `generare_meciuri` utilizează algoritmi de amestecare aleatorie (`std::shuffle`) pentru a crea perechi de jucători.

Clientul oferă o interfață text utilizatorului. Acesta implementează un meniu interactiv și rulează două procese paralele: unul care blochează pe `fgets` pentru inputul utilizatorului și unul care blochează pe `recv` pentru a afișa răspunsurile sau notificările asincrone primite de la server.

Mai jos este prezentată diagrama de secvență a interacțiunii pentru fluxul complet de creare și înscriere:

4 Protocolul de Comunicare

Comunicația este de tip text-based (ASCII). Mesajele sunt delimitate de caracterul newline. Comenzile implementate sunt:

1. `REGISTER <user> <pass> <mail>` - Creare cont nou. Salvează în `db_users.txt` cu flag-ul de admin 0 (implicit).
2. `LOGIN <user> <pass>` - Autentificare. Serverul verifică credențialele și încarcă drepturile utilizatorului în sesiune.
3. `ADAUGA_CAMP <nume> <joc> <nr> <tip>` - (Admin) Creează o competiție nouă.
4. `JOIN <id>` - Înscriere în campionat. Dacă numărul maxim de jucători este atins, serverul declanșează automat `generare_meciuri`.
5. `SET_SCOR <id_meci> <s1> <s2>` - Actualizare scor. Permisă doar participanților la meci sau adminilor.
6. `REPROGRAMEAZA <data>` - Modifică data de desfășurare a meciurilor utilizatorului curent.
7. `ISTORIC` - (Admin) Afișează toate meciurile din sistem.

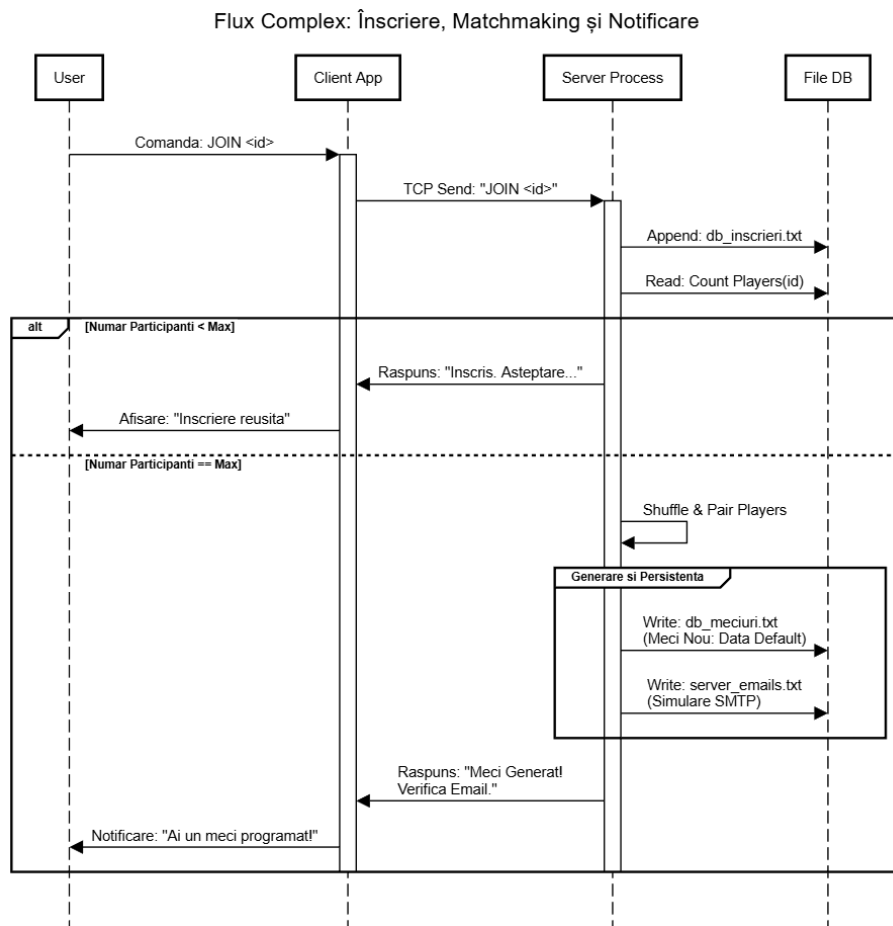


Fig.1. Diagrama de secvență: Fluxul de înregistrare, creare campionat și generare meciuri

5 Scenarii de Utilizare

5.1 Execuție cu Succes

1. **Flux Complet Jucător:** Utilizatorul se conectează, trimite **REGISTER**, primește confirmarea, apoi **LOGIN**. Solicită **LISTA** campionatelor și trimite **JOIN 1024**. Serverul confirmă înscrierea.
2. **Administrare:** Un utilizator cu drepturi de admin (setate manual în DB) se autentifică și trimite **ADAUGA.CAMP CupaIasi CS2 16 Single**. Serverul creează intrarea și răspunde cu ID-ul campionatului.
3. **Desfășurare Meci:** După generarea meciurilor, un jucător trimite **SET_SCOR 1 16 14**. Serverul validează că jucătorul face parte din meciul 1 și actualizează fișierul **db_meciuri.txt**.

5.2 Cazuri de Eșec

1. **Sesiune Invalidă:** Utilizatorul încearcă să dea `JOIN` sau `REPROGRAMEAZA` fără a fi logat. Serverul verifică variabila booleană `user.logat` și returnează eroarea "Trebuie sa fii logat."
2. **Sintaxă Greșită:** Se trimite comanda `REGISTER user` (fără parolă și mail). Serverul detectează lipsa argumentelor și trimite instrucțiuni de sintaxă.
3. **Acces Interzis:** Un utilizator obișnuit încearcă comanda `ISTORIC`. Serverul verifică flag-ul `user.este_admin` și respinge cererea ("Doar adminii pot vedea istoricul").

6 Concluzii

Soluția a demonstrat viabilitatea unui sistem de management distribuit folosind primitivele de sistem Linux și limbajul C++. Codul este modular, iar utilizarea TCP garantează integritatea comenzilor. Ca direcții viitoare, se recomandă criptarea parolelor (în prezent stocate plain-text), migrarea de la fișiere text la o bază de date SQL pentru performanță la volume mari de date și implementarea unui protocol SMTP real pentru trimiterea email-urilor.

7 Bibliografie

Andrew S. Tanenbaum, Nick Feamster, David Wetherall, Computer Networks, 6th Edition